HTML | CSS | XML | XHTML

Covers:

✔ Full Syllabus Coverage ✔ PYQ Answers

✔ Code Examples ✔ Comparison Tables

✔ Diagrams ✔ Quick Revision

✔ Viva & Interview Questions

B.Tech | Semester Examination Preparation

Textbooks: Jackson (Pearson, 2007) | Sebesta, 7th Ed. (Pearson, 2013)

Contents

1 HTML **5**

1.1 Introduction to HTML . . . 5
1.1.1 History of HTML . . . 5
1.1.2 Features of HTML . . . 5
1.1.3 Advantages and Limitations . . . 6
1.1.4 Static vs Dynamic Web Pages . . . 6
1.2 Basic HTML Syntax . . . 6
1.2.1 Tags, Elements and Attributes . . . 6
1.2.2 Global Attributes . . . 7
1.3 Standard HTML Document Structure . . . 7
1.3.1 Explanation of Each Part . . . 8
1.3.2 HTML Document Hierarchy Diagram . . . 8
1.4 Basic Text Markup . . . 8
1.4.1 Text Markup Tags Summary . . . 10
1.5 Headings (h1–h6) . . . 10
1.6 HTML Styles and Attributes . . . 10
1.6.1 The style Attribute (Inline CSS) . . . 10
1.7 Hyperlinks . . . 11
1.7.1 Attributes of the Anchor Tag . . . 11
1.8 Images . . . 12
1.8.1 Important Attributes of `<img>` . . . 13
1.9 HTML Layouts . . . 13
1.9.1 Semantic Layout (HTML5) . . . 13
1.9.2 Semantic Elements Summary . . . 14
1.10 IFrames . . . 14
1.11 Lists . . . 15
1.11.1 Types of Lists in HTML . . . 15
1.11.2 Comparison of List Types . . . 17
1.12 Tables . . . 17

1.12.1 Table Tags Summary 18
1.13 Forms 19
1.13.1 HTML Form Input Types 22
1.14 Form Validation with JavaScript 22
1.15 HTML Page with Frames/Hyperlinks (PYQ Program) 25
1.16 Dynamic HTML (DHTML) 26
1.16.1 Components of DHTML 26
1.16.2 Advantages of DHTML 27

2 CSS **28**
2.1 Introduction to CSS 28
2.1.1 Need for CSS 28
2.1.2 Why CSS over HTML Formatting? 28
2.2 Types of CSS (Style Sheets) 29
2.2.1 1. Inline CSS 29
2.2.2 2. Internal (Embedded) CSS 29
2.2.3 3. External CSS 30
2.2.4 Comparison of CSS Types 30
2.3 CSS Syntax and Structure 31
2.4 CSS Selectors 31
2.4.1 1. Universal Selector (*) 31
2.4.2 2. Element (Type) Selector 31
2.4.3 3. Class Selector (.classname) 32
2.4.4 4. ID Selector (#idname) 32
2.4.5 5. Group Selector (A, B, C) 33
2.4.6 6. Descendant Selector (A B) 33
2.4.7 7. Child Selector (A > B) 33
2.4.8 8. Adjacent Sibling Selector (A + B) 34
2.4.9 9. Attribute Selector 34
2.4.10 10. Pseudo-class Selectors 34
2.4.11 11. Pseudo-element Selectors 35
2.4.12 CSS Selectors Summary Table 35
2.5 Colors and Backgrounds 35
2.6 Text and Fonts 36
2.7 Borders, Margins and Padding 36
2.7.1 Difference: Margin vs Padding 37
2.8 The CSS Box Model 37
2.8.1 Box Model Diagram 38
2.8.2 Numerical Example 38

2.9 CSS Positioning . . . 39
2.10 CSS2 Features . . . 40

3 XML **41**
3.1 Introduction to XML . . . 41
3.1.1 Features of XML . . . 41
3.1.2 HTML vs XML . . . 41
3.2 XML Syntax . . . 42
3.2.1 XML Syntax Rules . . . 42
3.3 XML Namespaces . . . 43
3.4 DTD – Document Type Definition . . . 43
3.4.1 Internal DTD . . . 44
3.4.2 External DTD . . . 44
3.4.3 DTD Declarations . . . 45
3.5 XML Schema (XSD) . . . 45
3.5.1 DTD vs XML Schema . . . 45
3.6 DOM – Document Object Model . . . 46
3.6.1 DOM Tree Diagram . . . 46
3.7 XML Parsers . . . 46
3.7.1 DOM Parser . . . 47
3.7.2 SAX Parser . . . 47
3.7.3 DOM vs SAX Parser — Detailed Comparison . . . 47
3.8 XML Programs . . . 48
3.8.1 Student XML Document (PYQ Program) . . . 48

4 XHTML **52**
4.1 Introduction to XHTML . . . 52
4.1.1 Features of XHTML . . . 52
4.2 HTML vs XHTML . . . 52
4.3 Converting HTML to XHTML . . . 53
4.3.1 XHTML Conversion Rules . . . 54
4.4 Meta Tags . . . 54
4.5 Character Entities . . . 55
4.6 Frames and Framesets . . . 55
4.6.1 Frame Tag Attributes . . . 56

5 Previous Year Questions — Complete Answers **58**
5.1 HTML PYQs . . . 58
5.2 CSS PYQs . . . 59
5.3 XML / XHTML / DTD PYQs . . . 60

6 Viva & Interview Questions **62**
6.1 HTML Questions 62
6.2 CSS Questions 63
6.3 XML Questions 63
6.4 XHTML Questions 64

7 Unit-I Quick Revision **65**
7.1 Important HTML Tags 65
7.2 Important CSS Properties 65
7.3 Important XML Terms 66
7.4 Important XHTML Rules 66
7.5 Most Probable Exam Questions 66
7.6 One-Page Unit I Summary 67

Chapter 1

PART A : HyperText Markup Language (HTML)

1.1 Introduction to HTML

Definition

HTML (HyperText Markup Language) is the standard markup language used to create and structure content on the World Wide Web. It describes the structure of a web page using a series of *elements* represented by *tags*.

1.1.1 History of HTML

Version	Year	Key Features
HTML 1.0	1991	Basic tags; created by Tim Berners-Lee
HTML 2.0	1995	First standard; forms introduced
HTML 3.2	1997	Tables, applets, text flow around images
HTML 4.01	1999	CSS support, scripting, accessibility
XHTML 1.0	2000	XML-based HTML
HTML5	2014	Multimedia, semantic elements, APIs

1.1.2 Features of HTML

- Platform independent — runs on any OS/browser.
- Easy to learn and use.
- Supports multimedia: images, audio, video.
- Supports hyperlinks for navigation.

- Free and open standard (W3C).
- Case-insensitive (though lowercase is recommended).
- Supports forms for user input.

1.1.3 Advantages and Limitations

Advantages	Limitations
Simple syntax, easy to learn	Cannot produce dynamic content alone
Supported by all browsers	Limited design capabilities
Free to use	Security issues (no built-in security)
Large community and documentation	No logic/programming capability
SEO-friendly when structured well	Browser inconsistencies

1.1.4 Static vs Dynamic Web Pages

Static Web Pages	Dynamic Web Pages
Content fixed; same for all users	Content changes based on user/data
Written purely in HTML	Uses server-side (PHP, Python) or client-side (JS)
Fast loading	Slightly slower
No database needed	Usually needs database
Example: portfolio page	Example: social media, e-commerce

1.2 Basic HTML Syntax

1.2.1 Tags, Elements and Attributes

Key Terms

- **Tag**: A keyword enclosed in angle brackets, e.g. `<p>`.
- **Element**: An opening tag + content + closing tag, e.g. `<p>Hello</p>`.
- **Attribute**: Extra information inside the opening tag, e.g. `href`, `src`, `id`.
- **Empty Element**: A tag with no closing tag, e.g. `<br>`, `<hr>`, `<img>`.
- **Container Element**: Has both opening and closing tags, e.g. `<div>...</div>`.

```
<!-- Container element with attribute -->
```

```
<a href="https://www.example.com" target="_blank">Click Here
    </a>

<!-- Empty element -->
<img src="photo.jpg" alt="My Photo" width="200" height="150"
    >
<br>
<hr>

<!-- Nesting of tags -->
<p>This is <b>bold</b> and <i>italic</i> text.</p>
```

Listing 1.1: Anatomy of an HTML Element

1.2.2 Global Attributes

Attribute	Description
id	Unique identifier for an element
class	CSS class name(s)
style	Inline CSS styles
title	Tooltip text
lang	Language of the element's content
hidden	Hides the element
tabindex	Tab order for keyboard navigation
data-*	Custom data attributes

1.3 Standard HTML Document Structure

[PYQ] ★ [IMPORTANT]

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Web Technology Notes">
    <title>My Web Page</title>
    <link rel="stylesheet" href="style.css">
  </head>
```

```
  <body>
    <h1>Welcome to My Page</h1>
    <p>This is the body of the HTML document.</p>
    <script src="script.js"></script>
  </body>
</html>
```

Listing 1.2: Standard HTML5 Document Structure

1.3.1 Explanation of Each Part

Tag / Declaration	Purpose
`<!DOCTYPE html>`	Declares the document type as HTML5 to the browser
`<html>`	Root element; wraps the entire HTML page
`<head>`	Contains meta-information; not displayed on screen
`<title>`	Sets the browser tab title
`<meta>`	Metadata: charset, viewport, description, keywords
`<link>`	Links external stylesheets
`<body>`	All visible content of the page goes here
`<script>`	Links JavaScript files (usually at end of body)

1.3.2 HTML Document Hierarchy Diagram

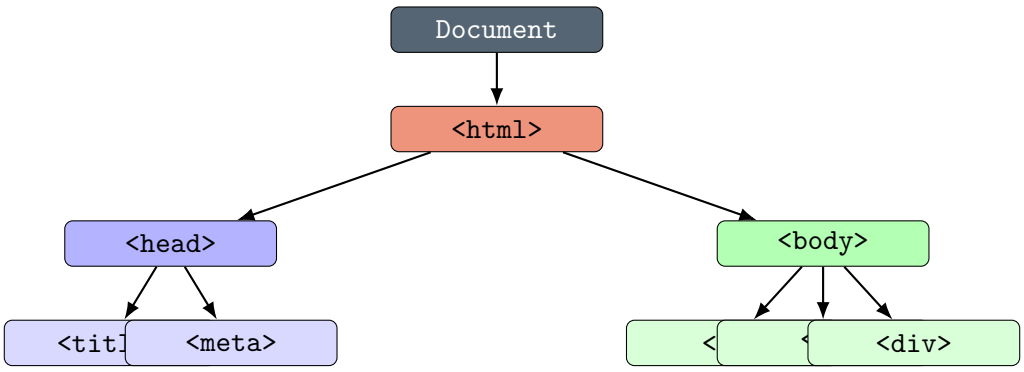


Figure 1.1: HTML Document Tree (DOM Hierarchy)

1.4 Basic Text Markup

```
<!DOCTYPE html>
<html>
<head><title>Text Markup</title></head>
<body>
```

```
5
6     <!-- Paragraphs -->
7     <p>This is a paragraph.</p>
8
9     <!-- Line Break -->
10    <p>Line one.<br>Line two (same paragraph).</p>
11
12    <!-- Horizontal Rule -->
13    <hr>
14
15    <!-- Bold and Strong -->
16    <b>Bold text</b>   <strong>Strong (bold + semantic)</
          strong>
17
18    <!-- Italic and Emphasis -->
19    <i>Italic text</i>   <em>Emphasized text</em>
20
21    <!-- Underline -->
22    <u>Underlined text</u>
23
24    <!-- Superscript and Subscript -->
25    <p>x<sup>2</sup> + y<sup>2</sup> = z<sup>2</sup></p>
26    <p>H<sub>2</sub>O is water</p>
27
28    <!-- Preformatted Text (preserves spaces and newlines) -->
29    <pre>
30      Name     : John
31      Roll No  : 101
32      Grade    : A
33    </pre>
34
35    <!-- Strikethrough -->
36    <s>Outdated content</s>
37
38    <!-- Inline code -->
39    <code>int x = 10;</code>
40
41  </body>
42  </html>
```

Listing 1.3: Text Markup Tags

1.4.1 Text Markup Tags Summary

Tag	Type	Purpose
`<p>`	Container	Paragraph
` `	Empty	Line break
`<hr>`	Empty	Horizontal rule
`<b>`	Container	Bold (presentational)
`<strong>`	Container	Bold + semantic importance
`<i>`	Container	Italic (presentational)
`<em>`	Container	Italic + semantic emphasis
`<u>`	Container	Underline
`<sup>`	Container	Superscript
`<sub>`	Container	Subscript
`<pre>`	Container	Preformatted text
`<s>`	Container	Strikethrough
`<code>`	Container	Inline code

1.5 Headings (h1–h6)

Note

HTML provides **six levels** of headings, `<h1>` being the largest (most important) and `<h6>` the smallest. Proper heading hierarchy improves SEO and accessibility.

```
<h1>Heading Level 1     Main Title</h1>
<h2>Heading Level 2     Chapter</h2>
<h3>Heading Level 3     Section</h3>
<h4>Heading Level 4     Subsection</h4>
<h5>Heading Level 5     Minor Heading</h5>
<h6>Heading Level 6     Smallest</h6>
```

Listing 1.4: All Heading Levels

1.6 HTML Styles and Attributes

1.6.1 The style Attribute (Inline CSS)

```
<p style="color:red; font-size:18px;">Red paragraph</p>
<h2 style="text-align:center; background-color:yellow;">
  Centered Heading
```

```
</h2>
<body style="background-color:#f0f0f0; font-family:Arial;">
```

Listing 1.5: style Attribute Examples

1.7 Hyperlinks

[PYQ] [IMPORTANT]

Anchor Tag Definition

The **Anchor Tag** <a> is used to create hyperlinks in HTML. It links one page to another or to a section within the same page.

Syntax: Link Text

```
<!-- 1. External Link -->
<a href="https://www.google.com" target="_blank">Visit
   Google</a>

<!-- 2. Internal Link (same site) -->
<a href="contact.html">Contact Us</a>

<!-- 3. Email Link -->
<a href="mailto:student@email.com">Send Email</a>

<!-- 4. Telephone Link -->
<a href="tel:+919876543210">Call Us</a>

<!-- 5. Bookmark (jump to section on same page) -->
<!-- Step 1: Mark the target -->
<h2 id="section2">Section 2</h2>
<!-- Step 2: Create the link -->
<a href="#section2">Go to Section 2</a>

<!-- 6. Download Link -->
<a href="notes.pdf" download>Download PDF Notes</a>
```

Listing 1.6: Hyperlink Types

1.7.1 Attributes of the Anchor Tag

Attribute	Values	Description
`href`	URL / #	Destination URL
`target`	_blank, _self, _parent	Where to open the link
`rel`	nofollow, noopener	Relationship to linked document
`download`	filename	Forces download
`title`	text	Tooltip on hover

1.8 Images

[PYQ]

Image Tag Definition

The **Image Tag** `<img>` is an empty (self-closing) tag used to embed images in a web page.

Syntax: `<img src="path" alt="description">`

```
<!-- Basic image -->
<img src="photo.jpg" alt="A beautiful photo">

<!-- Image with width and height -->
<img src="logo.png" alt="Company Logo" width="300" height="150">

<!-- Image as a link -->
<a href="index.html">
  <img src="home.png" alt="Home" width="50">
</a>

<!-- Image with title tooltip -->
<img src="sunset.jpg" alt="Sunset" title="Evening Sunset"
     style="border:2px solid black;">

<!-- Image with relative path -->
<img src="images/photo.jpg" alt="Profile Photo">

<!-- Image from URL -->
<img src="https://www.example.com/img.jpg" alt="Online Image">
```

Listing 1.7: Image Tag Examples

1.8.1 Important Attributes of <img>

Attribute	Example	Purpose
src	src="photo.jpg"	Path/URL to the image
alt	alt="Photo"	Alternate text (accessibility/SEO)
width	width="300"	Width in pixels or %
height	height="200"	Height in pixels or %
title	title="Tip"	Tooltip on mouse hover
style	style="..."	Inline CSS

1.9 HTML Layouts

1.9.1 Semantic Layout (HTML5)

HTML5 introduced **semantic elements** that describe the purpose of the content to both browser and developer.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    header { background:#2C3E50; color:white; padding:15px; }
    nav    { background:#1A5276; color:white; padding:10px; }
    section{ float:left; width:70%; padding:15px; }
    aside  { float:right; width:28%; padding:15px; }
    footer { background:#2C3E50; color:white;
             text-align:center; clear:both; padding:10px; }
  </style>
</head>
<body>
  <header> <h1>My Website</h1> </header>

  <nav>
    <a href="#">Home</a> |
    <a href="#">About</a> |
    <a href="#">Contact</a>
  </nav>

  <section>
```

```
    <article>
      <h2>Main Content</h2>
      <p>This is the main article content area.</p>
    </article>
  </section>

  <aside>
    <h3>Sidebar</h3>
    <p>Related links here.</p>
  </aside>

  <footer>
    <p>Copyright &copy; 2024 My Website</p>
  </footer>
</body>
</html>
```

Listing 1.8: Semantic Layout

1.9.2 Semantic Elements Summary

Element	Type	Purpose
`<header>`	Semantic	Page/section header
`<nav>`	Semantic	Navigation links
`<main>`	Semantic	Main content of the document
`<section>`	Semantic	Thematic grouping of content
`<article>`	Semantic	Independent, self-contained content
`<aside>`	Semantic	Sidebar / related content
`<footer>`	Semantic	Page/section footer
`<div>`	Non-semantic	Generic block-level container
`<span>`	Non-semantic	Generic inline container

1.10 IFrames

iFrame Definition

An **Inline Frame** (`<iframe>`) is used to embed another HTML document (or web page) within the current page.

```
<!-- Basic iframe -->
```

```html
<iframe src="page2.html" width="600" height="400"></iframe>

<!-- Embed external website -->
<iframe src="https://www.wikipedia.org"
        width="100%" height="500" frameborder="0">
</iframe>

<!-- Embed YouTube video -->
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/VIDEO_ID"
  allowfullscreen>
</iframe>

<!-- iFrame with name (target for links) -->
<iframe name="myframe" src="page1.html"
        width="700" height="400">
</iframe>
<a href="page2.html" target="myframe">Load Page 2</a>
```

Listing 1.9: iFrame Examples

Advantages	Disadvantages
Embeds external content easily	Security risks (clickjacking)
Isolates content	Can slow page loading
Useful for maps and videos	Not always search engine friendly
Independent scrolling	Inconsistent behavior across browsers

1.11 Lists

[PYQ]

1.11.1 Types of Lists in HTML

HTML supports three types of lists:

1. Ordered List `<ol>`

```html
<ol type="1"> <!-- type: 1, A, a, I, i -->
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
```

```
</ol>

<!-- Start from a specific number -->
<ol start="5">
  <li>Item five</li>
  <li>Item six</li>
</ol>
```

Listing 1.10: Ordered List

2. Unordered List

```
<ul style="list-style-type:disc"> <!-- disc/circle/square
    -->
  <li>HTML</li>
  <li>CSS</li>
  <li>XML</li>
</ul>
```

Listing 1.11: Unordered List

3. Description List <dl>

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language      used for web structure</
     dd>

  <dt>CSS</dt>
  <dd>Cascading Style Sheets      used for styling web pages
     </dd>

  <dt>XML</dt>
  <dd>Extensible Markup Language      used for data storage</
     dd>
</dl>
```

Listing 1.12: Description / Definition List

4. Nested Lists

```
<ol>
  <li>Web Technologies
```

```
    <ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
  </li>
  <li>Databases
    <ul>
      <li>MySQL</li>
      <li>MongoDB</li>
    </ul>
  </li>
</ol>
```

Listing 1.13: Nested List

1.11.2 Comparison of List Types

Feature	Ordered ( )	Unordered ( )	Description (<dl>)
Marker type	Numbers / Letters	Bullets	None
Use case	Sequential steps	Non-sequential	Term + definition
type attr.	1, A, a, I, i	disc, circle, square	N/A
Child tag			<dt>, <dd>

1.12 Tables

```
<table border="1" cellpadding="8" cellspacing="0"
       style="border-collapse:collapse; width:100%">

  <caption>Student Marks Table</caption>

  <!-- Table Header -->
  <thead>
    <tr style="background-color:#2C3E50; color:white;">
      <th>Roll No.</th>
      <th>Name</th>
      <th>Math</th>
      <th>Science</th>
      <th>Total</th>
```

```
    </tr>
  </thead>

  <!-- Table Body -->
  <tbody>
    <tr>
      <td>101</td>
      <td>Alice</td>
      <td>85</td>
      <td>90</td>
      <td>175</td>
    </tr>
    <tr>
      <td>102</td>
      <td>Bob</td>
      <td>78</td>
      <td>82</td>
      <td>160</td>
    </tr>
  </tbody>

  <!-- Table Footer -->
  <tfoot>
    <tr>
      <!-- colspan merges 4 columns into 1 -->
      <td colspan="4"><b>Class Average</b></td>
      <td>167.5</td>
    </tr>
  </tfoot>

</table>
```

Listing 1.14: Complete Table Example

1.12.1 Table Tags Summary

Tag	Full Name	Purpose
`<table>`	Table	Creates the table
`<tr>`	Table Row	Defines a row
`<th>`	Table Header	Header cell (bold + centred)

`<td>`	Table Data	Data cell
`<thead>`	Table Head	Groups header rows
`<tbody>`	Table Body	Groups body rows
`<tfoot>`	Table Footer	Groups footer rows
`<caption>`	Caption	Table title
`colspan`	Attribute	Merges columns horizontally
`rowspan`	Attribute	Merges rows vertically

1.13 Forms

[PYQ] ★ [IMPORTANT]

Form Tag Definition

The **Form Tag** `<form>` is used to create an HTML form that collects user input. The data is sent to a server for processing.

Syntax: `<form action="URL" method="GET|POST">...</form>`

```
<!DOCTYPE html>
<html>
<head>
  <title>Registration Form</title>
  <style>
    body { font-family: Arial; background: #f0f0f0; }
    .form-container {
      width: 500px; margin: 30px auto;
      background: white; padding: 30px;
      border-radius: 8px;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    }
    label  { display:block; margin-top:12px; font-weight:
       bold; }
    input, select, textarea {
      width:100%; padding:8px;
      border:1px solid #ccc; border-radius:4px;
      box-sizing: border-box;
    }
    .btn   { margin-top:18px; padding:10px 24px;
             background:#2C3E50; color:white;
             border:none; border-radius:4px; cursor:pointer;
                 }
```

```
22    </style>
23  </head>
24  <body>
25  <div class="form-container">
26    <h2>Student Registration</h2>
27
28    <form action="submit.php" method="POST" enctype="multipart
        /form-data">
29
30      <!-- Text Input -->
31      <label for="name">Full Name:</label>
32      <input type="text" id="name" name="name"
33             placeholder="Enter your full name" required>
34
35      <!-- Email Input -->
36      <label for="email">Email:</label>
37      <input type="email" id="email" name="email"
38             placeholder="example@email.com" required>
39
40      <!-- Password -->
41      <label for="password">Password:</label>
42      <input type="password" id="password" name="password"
43             placeholder="Min 8 characters" required>
44
45      <!-- Number -->
46      <label for="rollno">Roll Number:</label>
47      <input type="number" id="rollno" name="rollno"
48             min="1" max="9999" required>
49
50      <!-- Date -->
51      <label for="dob">Date of Birth:</label>
52      <input type="date" id="dob" name="dob">
53
54      <!-- Radio Buttons -->
55      <label>Gender:</label>
56      <input type="radio" id="male"   name="gender" value="
          Male">
57      <label for="male"   style="display:inline;">Male</label>
58      <input type="radio" id="female" name="gender" value="
          Female">
59      <label for="female" style="display:inline;">Female</
```

```
        label>
60      <input type="radio" id="other"  name="gender" value="
           Other">
61      <label for="other"  style="display:inline;">Other</label
           >
62
63      <!-- Checkbox -->
64      <label>Subjects Enrolled:</label>
65      <input type="checkbox" name="sub" value="HTML"> HTML
66      <input type="checkbox" name="sub" value="CSS">  CSS
67      <input type="checkbox" name="sub" value="JS">
           JavaScript
68
69      <!-- Dropdown / Select -->
70      <label for="course">Course:</label>
71      <select id="course" name="course">
72        <option value="">-- Select Course --</option>
73        <option value="btech">B.Tech CSE</option>
74        <option value="mtech">M.Tech</option>
75        <option value="bca">BCA</option>
76      </select>
77
78      <!-- Textarea -->
79      <label for="address">Address:</label>
80      <textarea id="address" name="address"
81                rows="3" placeholder="Enter address"></
                     textarea>
82
83      <!-- File Upload -->
84      <label for="photo">Upload Photo:</label>
85      <input type="file" id="photo" name="photo" accept="image
           /*">
86
87      <!-- Hidden Field -->
88      <input type="hidden" name="form_type" value="
           registration">
89
90      <!-- Submit and Reset Buttons -->
91      <button type="submit" class="btn">Register</button>
92      <button type="reset"  class="btn"
93              style="background:#c0392b;">Reset</button>
```

```
94
95    </form>
96  </div>
97  </body>
98  </html>
```

Listing 1.15: Complete Registration Form

1.13.1 HTML Form Input Types

Type	Description
`text`	Single-line text input
`password`	Masked text for passwords
`email`	Validates email format
`number`	Numeric input with optional min/max
`date`	Date picker
`radio`	Select one of several options
`checkbox`	Select multiple options
`file`	File upload control
`hidden`	Hidden data (not visible)
`submit`	Submit button
`reset`	Reset form fields
`button`	Generic clickable button
`range`	Slider control
`color`	Color picker
`url`	Validates URL format
`tel`	Telephone number input
`search`	Search field

1.14 Form Validation with JavaScript

[REPEATED] [PYQ] — *Repeated 3 times in university exams*

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Validated Registration Form</title>
5    <style>
6      body { font-family: Arial; margin: 30px; }
7      .error { color: red; font-size: 12px; }
```

```
    input  { width:300px; padding:6px; margin:4px 0; display
       :block; }
    button { padding:8px 20px; background:#2C3E50;
             color:white; border:none; cursor:pointer; }
  </style>
</head>
<body>
<h2>Student Registration Form</h2>

<form id="regForm" onsubmit="return validateForm()">

  <label>Name:
    <input type="text" id="name" placeholder="Enter Full
       Name">
    <span id="nameErr" class="error"></span>
  </label>

  <label>Email:
    <input type="email" id="email" placeholder="email@domain
       .com">
    <span id="emailErr" class="error"></span>
  </label>

  <label>Password:
    <input type="password" id="pwd" placeholder="Min 6 chars
       ">
    <span id="pwdErr" class="error"></span>
  </label>

  <br>
  <button type="submit">Register</button>

</form>

<script>
function validateForm() {

  var isValid = true;

  // Clear previous error messages
  document.getElementById("nameErr").innerHTML  = "";
```

```
45    document.getElementById("emailErr").innerHTML = "";
46    document.getElementById("pwdErr").innerHTML    = "";
47
48    // Validate Name
49    var name = document.getElementById("name").value.trim();
50    if (name === "") {
51      document.getElementById("nameErr").innerHTML =
52          "* Name is required";
53      isValid = false;
54    } else if (name.length < 3) {
55      document.getElementById("nameErr").innerHTML =
56          "* Name must be at least 3 characters";
57      isValid = false;
58    }
59
60    // Validate Email
61    var email = document.getElementById("email").value.trim();
62    var emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
63    if (email === "") {
64      document.getElementById("emailErr").innerHTML =
65          "* Email is required";
66      isValid = false;
67    } else if (!emailPattern.test(email)) {
68      document.getElementById("emailErr").innerHTML =
69          "* Enter a valid email address";
70      isValid = false;
71    }
72
73    // Validate Password
74    var pwd = document.getElementById("pwd").value;
75    if (pwd === "") {
76      document.getElementById("pwdErr").innerHTML =
77          "* Password is required";
78      isValid = false;
79    } else if (pwd.length < 6) {
80      document.getElementById("pwdErr").innerHTML =
81          "* Password must be at least 6 characters";
82      isValid = false;
83    }
84
85    if (isValid) {
```

```
    alert("Registration Successful!");
  }

  return isValid;
}
</script>

</body>
</html>
```

Listing 1.16: Student Registration Form with JavaScript Validation

1.15 HTML Page with Frames/Hyperlinks (PYQ Program)

[PYQ] — *Create an HTML page using frames/hyperlinks to display: Personal Information, Educational Information, Achievements, Experience*

```
<!DOCTYPE html>
<html>
<head>
  <title>My Profile</title>
  <style>
    body { font-family: Arial; background: #f5f5f5; }
    nav  { background: #2C3E50; padding: 10px; }
    nav a{ color: white; margin: 0 15px;
           text-decoration: none; font-size: 16px; }
    nav a:hover { text-decoration: underline; }
    .content { padding: 20px; }
  </style>
</head>
<body>
  <nav>
    <a href="personal.html"   target="main">Personal Info</a
       >
    <a href="education.html"  target="main">Education</a>
    <a href="achievements.html" target="main">Achievements</
       a>
    <a href="experience.html" target="main">Experience</a>
  </nav>
  <iframe name="main" src="personal.html"
```

```
            width="100%" height="500" frameborder="0">
  </iframe>
</body>
</html>
```

Listing 1.17: index.html — Navigation Page

```
<!DOCTYPE html>
<html>
<body>
  <h2>Personal Information</h2>
  <table border="1" cellpadding="8">
    <tr><td><b>Name</b></td>      <td>John Doe</td></tr>
    <tr><td><b>DOB</b></td>       <td>01-Jan-2000</td></tr>
    <tr><td><b>Address</b></td>   <td>New Delhi, India</td></
       tr>
    <tr><td><b>Phone</b></td>     <td>+91 9876543210</td></tr
       >
    <tr><td><b>Email</b></td>     <td>john@email.com</td></tr
       >
  </table>
</body>
</html>
```

Listing 1.18: personal.html

1.16 Dynamic HTML (DHTML)

[PYQ]

DHTML Definition

Dynamic HTML (DHTML) refers to the combination of HTML, CSS, JavaScript, and the DOM to create interactive and dynamic web content that can change after the page is loaded, without requiring a round-trip to the server.

1.16.1 Components of DHTML

Component	Role
HTML	Provides the structure and content
CSS	Controls presentation and visual styling

JavaScript	Adds interactivity and programming logic
DOM	Provides an API to access and modify HTML elements

```
<!DOCTYPE html>
<html>
<head>
  <style>
    #content { display:none; background:#dce; padding:10px;
       }
    button   { padding:8px 16px; cursor:pointer; }
  </style>
</head>
<body>
  <h2>DHTML Example</h2>
  <button onclick="toggleContent()">Show / Hide</button>
  <div id="content">
    <p>This content is shown/hidden using DHTML!</p>
  </div>

  <script>
    function toggleContent() {
      var el = document.getElementById("content");
      if (el.style.display === "none") {
        el.style.display = "block";
      } else {
        el.style.display = "none";
      }
    }
  </script>
</body>
</html>
```

Listing 1.19: DHTML Example – Show/Hide Content

1.16.2 Advantages of DHTML

- Reduces server load (changes happen client-side).
- Improves user experience with animations.
- Faster page updates (no full reload).
- Enables interactive forms and menus.

Chapter 2

PART B : Cascading Style Sheets (CSS)

2.1 Introduction to CSS

Definition

CSS (Cascading Style Sheets) is a style sheet language used to describe the presentation of an HTML document. CSS controls the layout, colours, fonts, spacing, and visual appearance of web pages.

2.1.1 Need for CSS

- Separates content (HTML) from presentation (CSS).
- Consistent styling across multiple pages.
- Easier maintenance — change one CSS file to update whole site.
- Faster loading (one CSS file cached by browser).
- Better accessibility and responsiveness.

2.1.2 Why CSS over HTML Formatting?

HTML Formatting	CSS
Style mixed with content	Style separate from content
Repeated on every element	Defined once, applied everywhere
Hard to maintain	Easy to maintain
Limited options	Full design control

Poor responsiveness	Supports responsive design

2.2 Types of CSS (Style Sheets)

[PYQ]

2.2.1 1. Inline CSS

Applied directly inside an HTML tag using the `style` attribute.

```
<p style="color:blue; font-size:18px; font-weight:bold;">
  This paragraph uses Inline CSS.
</p>
<h1 style="background-color:#2C3E50; color:white; padding:10
   px;">
  Inline Heading
</h1>
```

Listing 2.1: Inline CSS

2.2.2 2. Internal (Embedded) CSS

Defined inside the `<style>` tag within the `<head>` section.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body   { background-color: #f0f0f0; font-family: Arial;
       }
    h1     { color: #2C3E50; text-align: center; }
    p      { color: #333; line-height: 1.6; }
    .intro { background: #d0e8ff; padding: 10px; }
  </style>
</head>
<body>
  <h1>Internal CSS Example</h1>
  <p class="intro">This paragraph has a class applied.</p>
</body>
</html>
```

Listing 2.2: Internal CSS

2.2.3 3. External CSS

Styles are defined in a separate `.css` file and linked to the HTML.

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

Listing 2.3: Link external CSS in HTML

```
/* External CSS File */
body {
  background-color: #f9f9f9;
  font-family: 'Arial', sans-serif;
  margin: 0;
  padding: 0;
}

h1 {
  color: #2C3E50;
  text-align: center;
  border-bottom: 3px solid #E34C26;
  padding-bottom: 10px;
}

p {
  color: #555;
  line-height: 1.8;
  font-size: 16px;
}
```

Listing 2.4: styles.css

2.2.4 Comparison of CSS Types

Feature	Inline	Internal	External
Location	Inside HTML tag	`<head>` section	Separate .css file
Scope	Single element	Single page	Multiple pages
Maintenance	Difficult	Moderate	Easy
Reusability	None	Page only	Across website
Priority	Highest	Medium	Lowest

Best for	Quick fixes	Single-page apps	Large websites

2.3 CSS Syntax and Structure

CSS Syntax

```
selector {
    property: value;
    property: value;
}
```

- **Selector**: Targets the HTML element(s) to style.
- **Property**: The CSS property to change (e.g. `color`).
- **Value**: The value for that property (e.g. `red`).
- **Declaration Block**: `{ property: value; }`

2.4 CSS Selectors

[REPEATED] [IMPORTANT] — *Repeated 3+ times in university exams*

Important / Repeated PYQ

CSS Selectors are patterns used to select the elements you want to style. This is one of the most frequently asked topics in university exams.

2.4.1 1. Universal Selector (*)

Selects *all* elements on the page.

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

Listing 2.5: Universal Selector

2.4.2 2. Element (Type) Selector

Selects all elements of a specific HTML tag.

```
p {
    color: #333;
```

```
    font-size: 16px;
}
h1 {
    color: navy;
    font-family: Arial;
}
```

Listing 2.6: Element Selector

2.4.3 3. Class Selector (.classname)

Selects all elements with a specific class attribute.

```
.highlight {
    background-color: yellow;
    font-weight: bold;
}
.card {
    border: 1px solid #ccc;
    padding: 15px;
    border-radius: 8px;
}
```

Listing 2.7: Class Selector

```
<p class="highlight">This text is highlighted.</p>
<div class="card">Card content here.</div>
```

2.4.4 4. ID Selector (#idname)

Selects a *single* element with a specific ID (unique per page).

```
#header {
    background-color: #2C3E50;
    color: white;
    padding: 20px;
}
#main-content {
    max-width: 900px;
    margin: auto;
}
```

Listing 2.8: ID Selector

```html
<div id="header">Welcome</div>
<div id="main-content">Content</div>
```

2.4.5 5. Group Selector (A, B, C)

Applies the same styles to multiple selectors separated by commas.

```css
h1, h2, h3 {
    font-family: Georgia, serif;
    color: #1A5276;
}

p, li, dd {
    line-height: 1.6;
    color: #444;
}
```

Listing 2.9: Group Selector

2.4.6 6. Descendant Selector (A B)

Selects elements that are *descendants* of another element.

```css
div p {
    color: blue;  /* all <p> inside any <div> */
}

nav a {
    text-decoration: none;
    color: white;
}
```

Listing 2.10: Descendant Selector

2.4.7 7. Child Selector (A > B)

Selects only *direct children* of an element.

```css
ul > li {
    list-style: square; /* direct <li> of <ul> only */
}

div > p {
```

```
    font-weight: bold;  /* direct <p> children only */
}
```

Listing 2.11: Child Selector

2.4.8 8. Adjacent Sibling Selector (A + B)

Selects an element that is *immediately after* another element.

```
h1 + p {
    font-size: 18px;  /* first <p> right after <h1> */
    color: gray;
}
```

Listing 2.12: Adjacent Sibling Selector

2.4.9 9. Attribute Selector

Selects elements based on their attributes.

```
input[type="text"]  { border: 2px solid blue; }
input[type="email"] { border: 2px solid green; }
a[target="_blank"]  { color: red; }   /* external links */
img[alt]            { border: 1px solid gray; }
```

Listing 2.13: Attribute Selectors

2.4.10 10. Pseudo-class Selectors

Select elements in a specific *state*.

```
a:link    { color: blue; }     /* unvisited link */
a:visited { color: purple; }   /* visited link */
a:hover   { color: red; }      /* mouse over */
a:active  { color: orange; }   /* clicking */

input:focus { outline: 2px solid blue; }

li:first-child { font-weight: bold; }
li:last-child  { color: gray; }
li:nth-child(2){ background: yellow; }

p:not(.intro)  { font-size: 14px; }
```

Listing 2.14: Pseudo-class Selectors

2.4.11 11. Pseudo-element Selectors

Select and style a specific *part* of an element.

```
p::first-line   { font-weight: bold; color: navy; }
p::first-letter { font-size: 36px; float: left; }
h1::before      { content: ">> "; color: red; }
h1::after       { content: " <<"; color: red; }
```

Listing 2.15: Pseudo-element Selectors

2.4.12 CSS Selectors Summary Table

Selector Type	Syntax	Example
Universal	*	* { margin:0; }
Element	tag	p { color:red; }
Class	.name	.box { border:1px; }
ID	#name	#header { bg:blue; }
Group	A, B	h1,h2 { font:..}
Descendant	A B	div p { color:b; }
Child	A > B	ul > li { ..}
Adjacent	A + B	h1+p { ..}
Attribute	[attr=val]	input[type="text"]
Pseudo-class	:state	a:hover
Pseudo-element	::part	p::first-line

2.5 Colors and Backgrounds

```
/* Named color */
body { background-color: lightblue; }

/* HEX color */
h1   { color: #E34C26; }

/* RGB color */
p    { color: rgb(44, 62, 80); }

/* RGBA (with transparency) */
div  { background-color: rgba(44, 62, 80, 0.5); }

/* Background Image */
```

```css
body {
    background-image: url("bg.jpg");
    background-repeat: no-repeat;
    background-size: cover;
    background-position: center;
}
```

Listing 2.16: Colors and Background

2.6 Text and Fonts

```css
p {
    font-family: 'Arial', Helvetica, sans-serif;
    font-size: 16px;           /* size of text */
    font-weight: bold;         /* bold / normal / 100-900 */
    font-style: italic;        /* italic / normal / oblique */
    line-height: 1.8;          /* spacing between lines */
    letter-spacing: 1px;       /* spacing between letters */
    text-align: justify;       /* left|right|center|justify */
    text-decoration: underline; /* underline|overline|none */
    text-transform: uppercase;  /* uppercase|lowercase|capitalize */
    text-indent: 20px;          /* indent first line */
    color: #333;
    word-spacing: 3px;
}
```

Listing 2.17: Text and Font Properties

2.7 Borders, Margins and Padding

```css
div {
    /* Border */
    border: 2px solid #2C3E50;
    border-radius: 8px;
    border-top: 4px dashed red;   /* specific side */

    /* Margin (outside the element) */
```

```
    margin: 20px;            /* all sides */
    margin-top: 10px;
    margin-right: 15px;
    margin-bottom: 10px;
    margin-left: 15px;
    /* shorthand: top right bottom left */
    margin: 10px 15px 10px 15px;

    /* Padding (inside the element) */
    padding: 15px;           /* all sides */
    padding-top: 10px;
    padding-right: 20px;
    padding-bottom: 10px;
    padding-left: 20px;
}
```

Listing 2.18: Border, Margin and Padding

2.7.1 Difference: Margin vs Padding

Margin	Padding
Space *outside* the border	Space *inside* the border
Does not have background colour	Inherits background colour of element
Used for spacing between elements	Used for inner spacing
Can be negative	Cannot be negative
Part of Box Model (outer)	Part of Box Model (inner)

2.8 The CSS Box Model

[REPEATED] ★ [IMPORTANT] — *One of the most repeated CSS topics*

Box Model Definition

Every HTML element is treated as a **rectangular box**. The CSS Box Model describes how the element's dimensions are calculated from inside to outside: **Content → Padding → Border → Margin**.

2.8.1 Box Model Diagram

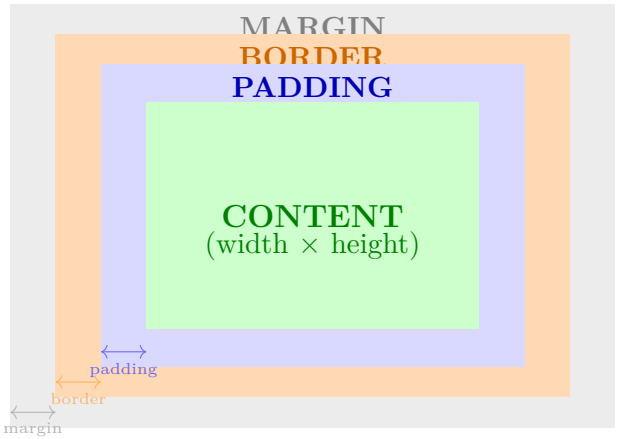


Figure 2.1: CSS Box Model

2.8.2 Numerical Example

```
div {
    width:  300px;    /* content width */
    height: 150px;    /* content height */
    padding: 20px;
    border: 5px solid black;
    margin: 10px;
}

/* Total width  = 300 + 20+20 (pad) + 5+5 (border) = 350px
   */
/* Total height = 150 + 20+20 (pad) + 5+5 (border) = 200px
   */
/* Margin adds space OUTSIDE but not to total size */

/* With box-sizing: border-box; */
/* width (300px) INCLUDES padding + border */
div {
    box-sizing: border-box;
    /* Now total width = 300px (padding+border counted
       inside) */
}
```

Listing 2.19: Box Model Calculation

2.9 CSS Positioning

[PYQ]

Value	Description	Example
`static`	Default; follows normal document flow	N/A
`relative`	Positioned relative to its normal position	Move 10px right
`absolute`	Positioned relative to nearest positioned ancestor	Popups, dropdowns
`fixed`	Fixed to viewport; doesn't scroll	Sticky navbar
`sticky`	Relative then fixed when scrolled past threshold	Scrolling headers

```
/* Relative */
.box-rel {
    position: relative;
    top: 20px;
    left: 30px;
}

/* Absolute */
.parent { position: relative; }
.child  {
    position: absolute;
    top: 10px;
    right: 10px;
}

/* Fixed (stays on screen while scrolling) */
.navbar {
    position: fixed;
    top: 0; left: 0;
    width: 100%;
    background: #2C3E50;
    z-index: 1000;
}

```

```css
/* Sticky */
.header {
    position: sticky;
    top: 0;
    background: white;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2);
}
```

Listing 2.20: Positioning Examples

2.10 CSS2 Features

CSS2 (1998) introduced significant improvements:

- **Media types**: different styles for print, screen, etc.
- **Advanced selectors**: attribute, pseudo-elements.
- **Positioning**: absolute, relative, fixed, z-index.
- **Aural style sheets**: for text-to-speech.
- **Counters**: auto-numbering using CSS.
- **Generated content**: `::before`, `::after`.
- **Table layout**: control over table rendering.

Chapter 3

PART C : Extensible Markup Language (XML)

3.1 Introduction to XML

[REPEATED] [PYQ] — *"What is XML and why is it used?" Repeated 3 times*

Definition

XML (Extensible Markup Language) is a markup language designed to *store and transport data* in a format that is both human-readable and machine-readable. Unlike HTML, XML has no predefined tags — you define your own tags to describe data.

3.1.1 Features of XML

- **Self-descriptive**: Tags describe the meaning of data.
- **Extensible**: Users define their own tags.
- **Platform-independent**: Works on any OS/language.
- **Strict syntax**: Must be well-formed.
- **Unicode support**: Supports all human languages.
- **Tree structure**: Hierarchical data representation.

3.1.2 HTML vs XML

[PYQ] — *Repeated twice in exams*

HTML	XML

HyperText Markup Language	Extensible Markup Language
Displays data	Stores and transports data
Predefined tags	User-defined tags
Not case-sensitive	Case-sensitive
Closing tags optional for some	All tags must be closed
Focus on presentation	Focus on data description
Fixed structure	Flexible structure
Browser renders it	No default rendering
Example: `<b>` makes bold	Example: `<name>` stores name

3.2 XML Syntax

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- This is an XML comment -->
<bookstore>
  <book category="fiction">
    <title>The Great Gatsby</title>
    <author>F. Scott Fitzgerald</author>
    <year>1925</year>
    <price>299.00</price>
  </book>
  <book category="science">
    <title>A Brief History of Time</title>
    <author>Stephen Hawking</author>
    <year>1988</year>
    <price>399.00</price>
  </book>
</bookstore>
```

Listing 3.1: Well-formed XML Document

3.2.1 XML Syntax Rules

1. Every XML document must have a **root element**.
2. All tags must be **properly closed**: `<tag></tag>` or `<tag/>`.
3. Tags are **case-sensitive**: `<Name>` $\neq$ `<name>`.
4. Tags must be **properly nested**.
5. Attribute values must be **quoted**.
6. Special characters must use **entities**: `<` for `<`.

3.3 XML Namespaces

[REPEATED] [PYQ] — *Repeated twice in exams*

Definition

XML Namespaces provide a way to avoid element name conflicts when combining XML documents from different vocabularies. They are declared using the `xmlns` attribute.

Syntax: `xmlns:prefix="URI"`

```xml
<?xml version="1.0" encoding="UTF-8"?>
<root
  xmlns:student="http://www.university.edu/student"
  xmlns:course="http://www.university.edu/course">

  <student:name>John Doe</student:name>
  <student:roll>2021CSE001</student:roll>

  <course:name>Web Technology</course:name>
  <course:code>CS401</course:code>

</root>
```

Listing 3.2: XML Namespace Example

```xml
<!-- Default namespace (no prefix needed) -->
<html xmlns="http://www.w3.org/1999/xhtml">
  <head><title>XHTML Page</title></head>
  <body><p>Hello XHTML</p></body>
</html>
```

Listing 3.3: Default Namespace

3.4 DTD – Document Type Definition

[PYQ]

Definition

A **DTD (Document Type Definition)** defines the legal structure of an XML document. It specifies which elements are allowed, their attributes, their order, and their nesting. DTD is used to *validate* XML documents.

3.4.1 Internal DTD

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE student [
  <!ELEMENT student (name, roll, marks)>
  <!ELEMENT name    (#PCDATA)>
  <!ELEMENT roll    (#PCDATA)>
  <!ELEMENT marks   (#PCDATA)>
  <!ATTLIST student grade CDATA #REQUIRED>
]>
<student grade="A">
  <name>Alice Johnson</name>
  <roll>101</roll>
  <marks>485</marks>
</student>
```

Listing 3.4: Internal DTD Example

3.4.2 External DTD

```
<!ELEMENT students (student+)>
<!ELEMENT student  (name, roll, marks+, total, percentage)>
<!ELEMENT name     (#PCDATA)>
<!ELEMENT roll     (#PCDATA)>
<!ELEMENT marks    (#PCDATA)>
<!ELEMENT total    (#PCDATA)>
<!ELEMENT percentage (#PCDATA)>
<!ATTLIST marks subject CDATA #REQUIRED>
```

Listing 3.5: External DTD — student.dtd

```
<?xml version="1.0"?>
<!DOCTYPE students SYSTEM "student.dtd">
<students>
  <student>
    <name>Bob Smith</name>
    <roll>102</roll>
    <marks subject="Maths">85</marks>
    <total>425</total>
    <percentage>85.00</percentage>
  </student>
</students>
```

Listing 3.6: XML referencing external DTD

3.4.3 DTD Declarations

Declaration	Meaning
`<!ELEMENT name (#PCDATA)>`	Element contains parsed character data (text)
`<!ELEMENT parent (child+)>`	Parent has one or more child elements
`<!ELEMENT parent (child*)>`	Parent has zero or more child elements
`<!ELEMENT parent (child?)>`	Parent has zero or one child element
`<!ATTLIST elem attr CDATA #REQUIRED>`	Attribute is required
`<!ATTLIST elem attr CDATA #IMPLIED>`	Attribute is optional
`<!ATTLIST elem attr CDATA #FIXED "val">`	Attribute has a fixed value

3.5 XML Schema (XSD)

Definition

XML Schema (XSD) is a more powerful alternative to DTD for defining the structure and data types in an XML document. XSD is itself written in XML.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="student">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name"  type="xs:string"/>
        <xs:element name="roll"  type="xs:integer"/>
        <xs:element name="email" type="xs:string"/>
        <xs:element name="marks" type="xs:integer"
                    minOccurs="1" maxOccurs="5"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

Listing 3.7: Simple XML Schema

3.5.1 DTD vs XML Schema

DTD	XML Schema (XSD)
Not written in XML syntax	Written in XML syntax
Limited data types	Rich data types (integer, date, etc.)
No namespace support	Full namespace support
Older, simpler	Newer, more powerful
Less flexible	More flexible and extensible

3.6 DOM – Document Object Model

Definition

The **DOM (Document Object Model)** is a platform-independent interface that represents an XML or HTML document as a *tree of objects*. Each element, attribute, and text in the document is a *node* in this tree.

3.6.1 DOM Tree Diagram

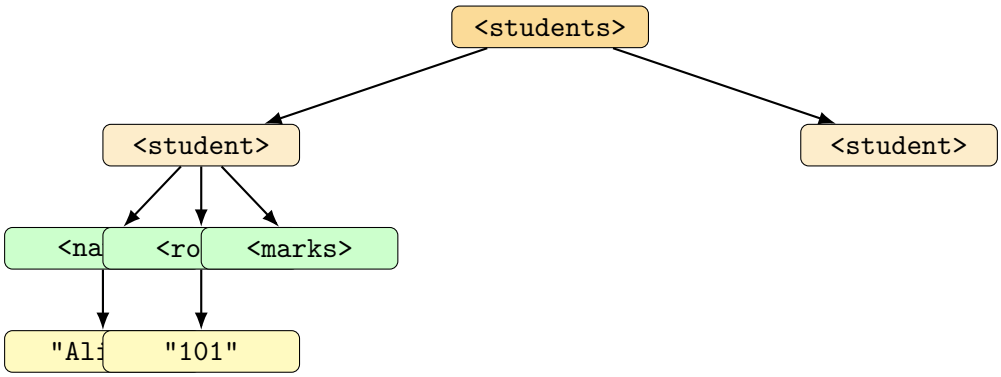


Figure 3.1: XML DOM Tree Structure

3.7 XML Parsers

[REPEATED] ★ [IMPORTANT] — *"Compare DOM Parser and SAX Parser"*
Repeated 3 times

3.7.1 DOM Parser

DOM Parser

The **DOM Parser** reads the *entire XML document* into memory and builds a **complete tree structure**. Nodes can be accessed, modified, added or deleted *in any order*.

Working of DOM Parser:

1. Reads the entire XML file.
2. Builds a tree (DOM Tree) in memory.
3. Application traverses/modifies the tree.

3.7.2 SAX Parser

SAX Parser

The **SAX (Simple API for XML) Parser** reads the XML document *sequentially* from top to bottom. It fires **events** (startElement, endElement, characters) as it reads each part. It does NOT build a tree in memory.

Working of SAX Parser:

1. Reads XML file line by line.
2. Fires event on encountering each element.
3. Handler methods process each event.
4. Does not store the full document in memory.

3.7.3 DOM vs SAX Parser — Detailed Comparison

Feature	DOM Parser	SAX Parser
Full name	Document Object Model	Simple API for XML
Approach	Tree-based	Event-based
Memory	Loads entire document	Low memory; no tree
Speed	Slower (tree build)	Faster (stream)
Direction	Random access	Sequential (top-down)
Modification	Can modify document	Read-only
Complexity	Easier to use	Harder to program

Best for	Small/medium XML, editing	Large XML, read-only

3.8 XML Programs

3.8.1 Student XML Document (PYQ Program)

[REPEATED] [PYQ] — *Repeated twice in university exams*

```xml
<?xml version="1.0" encoding="UTF-8"?>
<students>

  <student>
    <rollno>2021001</rollno>
    <name>Alice Johnson</name>
    <marks subject="Maths">85</marks>
    <marks subject="Physics">78</marks>
    <marks subject="Chemistry">90</marks>
    <marks subject="English">88</marks>
    <marks subject="CS">95</marks>
    <total>436</total>
    <percentage>87.20</percentage>
  </student>

  <student>
    <rollno>2021002</rollno>
    <name>Bob Smith</name>
    <marks subject="Maths">72</marks>
    <marks subject="Physics">65</marks>
    <marks subject="Chemistry">70</marks>
    <marks subject="English">80</marks>
    <marks subject="CS">88</marks>
    <total>375</total>
    <percentage>75.00</percentage>
  </student>

  <student>
    <rollno>2021003</rollno>
    <name>Carol White</name>
    <marks subject="Maths">90</marks>
    <marks subject="Physics">85</marks>
    <marks subject="Chemistry">88</marks>
```

```
    <marks subject="English">92</marks>
    <marks subject="CS">96</marks>
    <total>451</total>
    <percentage>90.20</percentage>
  </student>

  <student>
    <rollno>2021004</rollno>
    <name>David Brown</name>
    <marks subject="Maths">60</marks>
    <marks subject="Physics">55</marks>
    <marks subject="Chemistry">65</marks>
    <marks subject="English">70</marks>
    <marks subject="CS">75</marks>
    <total>325</total>
    <percentage>65.00</percentage>
  </student>

  <student>
    <rollno>2021005</rollno>
    <name>Emma Davis</name>
    <marks subject="Maths">88</marks>
    <marks subject="Physics">82</marks>
    <marks subject="Chemistry">79</marks>
    <marks subject="English">85</marks>
    <marks subject="CS">91</marks>
    <total>425</total>
    <percentage>85.00</percentage>
  </student>

  <student>
    <rollno>2021006</rollno>
    <name>Frank Wilson</name>
    <marks subject="Maths">45</marks>
    <marks subject="Physics">50</marks>
    <marks subject="Chemistry">48</marks>
    <marks subject="English">55</marks>
    <marks subject="CS">60</marks>
    <total>258</total>
    <percentage>51.60</percentage>
  </student>
```

```
75
76    <student>
77      <rollno>2021007</rollno>
78      <name>Grace Lee</name>
79      <marks subject="Maths">95</marks>
80      <marks subject="Physics">93</marks>
81      <marks subject="Chemistry">91</marks>
82      <marks subject="English">97</marks>
83      <marks subject="CS">99</marks>
84      <total>475</total>
85      <percentage>95.00</percentage>
86    </student>
87
88    <student>
89      <rollno>2021008</rollno>
90      <name>Henry Taylor</name>
91      <marks subject="Maths">70</marks>
92      <marks subject="Physics">68</marks>
93      <marks subject="Chemistry">72</marks>
94      <marks subject="English">75</marks>
95      <marks subject="CS">80</marks>
96      <total>365</total>
97      <percentage>73.00</percentage>
98    </student>
99
100   <student>
101     <rollno>2021009</rollno>
102     <name>Iris Martinez</name>
103     <marks subject="Maths">82</marks>
104     <marks subject="Physics">79</marks>
105     <marks subject="Chemistry">84</marks>
106     <marks subject="English">86</marks>
107     <marks subject="CS">89</marks>
108     <total>420</total>
109     <percentage>84.00</percentage>
110   </student>
111
112   <student>
113     <rollno>2021010</rollno>
114     <name>Jack Anderson</name>
115     <marks subject="Maths">55</marks>
```

```
116     <marks subject="Physics">60</marks>
117     <marks subject="Chemistry">58</marks>
118     <marks subject="English">62</marks>
119     <marks subject="CS">65</marks>
120     <total>300</total>
121     <percentage>60.00</percentage>
122   </student>
123
124 </students>
```

Listing 3.8: 10 Students with Roll No, Marks in 5 Subjects, Total, Percentage

Chapter 4

PART D : XHTML and Related Concepts

4.1 Introduction to XHTML

Definition

XHTML (Extensible HyperText Markup Language) is a reformulation of HTML 4.01 using the stricter rules of XML. XHTML combines the presentation capabilities of HTML with the strictness of XML.

4.1.1 Features of XHTML

- All tags must be in **lowercase**.
- All tags must be **properly closed**.
- Tags must be **properly nested**.
- Attribute values must be **quoted**.
- Empty elements need a **closing slash**: `<br />`, `<img />`.
- Every XHTML document must have a **DOCTYPE declaration**.

4.2 HTML vs XHTML

[PYQ] [IMPORTANT]

HTML	XHTML
Tags case-insensitive	All tags must be lowercase
Closing tags optional (` `)	All tags must be closed (` `)

Attribute values need not be quoted	Attribute values must be quoted
Nested incorrectly allowed	Must be properly nested
DOCTYPE optional	DOCTYPE is mandatory
Loosely parsed	Strictly parsed as XML
Forgiving of errors	Strict; errors stop rendering

4.3 Converting HTML to XHTML

[PYQ]

```
<!-- HTML - loose syntax -->
<HTML>
<HEAD><TITLE>My Page</TITLE></HEAD>
<BODY>
<H1>Hello World</H1>
<P>This is a paragraph
<BR>
<IMG SRC=photo.jpg ALT=Photo>
<INPUT TYPE=text NAME=user>
</BODY>
</HTML>
```

Listing 4.1: HTML (before)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head><title>My Page</title></head>
<body>
  <h1>Hello World</h1>
  <p>This is a paragraph</p>
  <br />
  <img src="photo.jpg" alt="Photo" />
  <input type="text" name="user" />
</body>
</html>
```

Listing 4.2: XHTML (after conversion)

4.3.1 XHTML Conversion Rules

1. All tag names must be in **lowercase**.
2. All tags must be **closed** (including `<br />`, `<hr />`, `<img />`).
3. Tags must be **correctly nested**.
4. All attribute values must be **in quotes**.
5. Attributes must have explicit values (no minimisation).
6. Document must have a **valid DOCTYPE**.
7. Root element `<html>` must declare the **XHTML namespace**.

4.4 Meta Tags

```
<head>
  <!-- Character encoding -->
  <meta charset="UTF-8">

  <!-- Viewport for responsive design -->
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">

  <!-- Page description (used by search engines) -->
  <meta name="description"
        content="Complete notes for Web Technology Unit 1">

  <!-- Keywords (for SEO) -->
  <meta name="keywords"
        content="HTML, CSS, XML, XHTML, Web Technology">

  <!-- Author -->
  <meta name="author" content="John Doe">

  <!-- Auto-refresh every 30 seconds -->
  <meta http-equiv="refresh" content="30">

  <!-- Redirect after 5 seconds -->
  <meta http-equiv="refresh"
        content="5; url=https://www.newpage.com">
</head>
```

Listing 4.3: Common Meta Tags

4.5 Character Entities

Note

HTML Character Entities are special codes used to display reserved or special characters in HTML/XHTML. They start with `&` and end with `;`.

Character	Entity Name	Entity Number	Description
<	`<`	`<`	Less than
>	`>`	`>`	Greater than
&	`&`	`&`	Ampersand
"	`"`	`"`	Double quote
‘	`'`	`'`	Apostrophe
(space)	` `	` `	Non-breaking space
©	`©`	`©`	Copyright
®	`®`	`®`	Registered
×	`×`	`×`	Multiplication

```
<p>5 &lt; 10 and 10 &gt; 5</p>
<p>Copyright &copy; 2024 My Company&reg;</p>
<p>Price: 100 &times; 5 = 500</p>
<p>Use    for non-breaking spaces</p>
<p>He said &quot;Hello World&quot;</p>
```

Listing 4.4: Using Character Entities

4.6 Frames and Framesets

[PYQ]

Note

Note: Frames (`<frameset>`) were part of HTML 4 but are *deprecated in HTML5*. XHTML still supports them. Use `<iframe>` for modern frame-like functionality.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN"
  "http://www.w3.org/TR/html4/frameset.dtd">
```

```
<html>
<head><title>Frames Example</title></head>

<!-- Two rows: top 20%, bottom 80% -->
<frameset rows="20%,80%">

  <!-- Top frame: navigation bar -->
  <frame src="topnav.html" name="topFrame"
         scrolling="no" noresize>

  <!-- Bottom row split into two columns: 25% + 75% -->
  <frameset cols="25%,75%">
    <frame src="sidenav.html" name="leftFrame">
    <frame src="content.html" name="mainFrame">
  </frameset>

  <noframes>
    <body>
      <p>Your browser does not support frames.
         Please use a modern browser.</p>
    </body>
  </noframes>

</frameset>
</html>
```

Listing 4.5: Frameset with Rows and Columns

```
<!DOCTYPE html>
<html>
<head><title>Percentage Frames</title></head>
<frameset cols="30%,40%,30%">
  <frame src="left.html"   name="leftFrame">
  <frame src="center.html" name="centerFrame">
  <frame src="right.html"  name="rightFrame">
</frameset>
</html>
```

Listing 4.6: Percentage-based Frame (PYQ Program)

4.6.1 Frame Tag Attributes

Attribute	Values	Description
`src`	URL	Source file for the frame
`name`	text	Name for targeting with links
`scrolling`	yes/no/auto	Scrollbar control
`noresize`	—	Prevents resizing
`frameborder`	0 or 1	Shows/hides frame border
`rows`	px/%	Row heights in frameset
`cols`	px/%	Column widths in frameset

Chapter 5

Previous Year Questions — Complete Answers

5.1 HTML PYQs

PYQ – Frequently Asked

Q1. State the difference between HTML and XML. [REPEATED]

Answer: See Section 3.1 (HTML vs XML comparison table).

PYQ – Frequently Asked

Q2. State the difference between XHTML and HTML. [PYQ]

Answer: See Section 4.2 (HTML vs XHTML table).

PYQ – Frequently Asked

Q3. What is HTML? Explain the structure of an HTML webpage with an example.

Answer: HTML (HyperText Markup Language) is the standard markup language for creating web pages. See Section 1.1 for the definition and Section 1.3 for the document structure with a complete example.

PYQ – Frequently Asked

Q4. Define Anchor Tag and Image Tag with examples.

Answer: See Section 1.7 (Anchor Tag) and Section 1.8 (Image Tag) with full code examples.

PYQ – Frequently Asked

Q5. Create a student registration form in HTML and validate the name and email fields using JavaScript. [REPEATED] (Repeated 3 times)

Answer: See Section 1.13 (Complete Registration Form) and Section 1.14 (JavaScript Validation Example).

PYQ – Frequently Asked

Q6. What are the types of lists in HTML? Explain with code.

Answer: HTML has three types: (1) Ordered List `<ol>`, (2) Unordered List `<ul>`, (3) Description List `<dl>`. See Section 1.11 for complete examples of all types including nested lists.

PYQ – Frequently Asked

Q7. Write a program to create a frame using percentage (%).

Answer: See Section 4.6 — Percentage-based Frame listing.

PYQ – Frequently Asked

Q8. What are XHTML and DHTML?

Answer:

- **XHTML**: See Section 4.1 — Reformulation of HTML using XML rules.
- **DHTML**: See Section 1.15 — Combination of HTML, CSS, JS, and DOM for dynamic content.

5.2 CSS PYQs

PYQ – Frequently Asked

Q9. Discuss CSS Selectors in detail. [REPEATED] (Repeated 3 times)

Answer: See Section 2.4 — All 11 types of CSS selectors with detailed explanation, syntax, and code examples.

PYQ – Frequently Asked

Q10. Describe the CSS Box Model with example. [REPEATED] (Repeated multiple times)

Answer: See Section 2.8 — Box Model (Content, Padding, Border, Margin) with diagram and numerical calculation.

PYQ – Frequently Asked

Q11. What is the difference between Margin and Padding?

Answer: See Section 2.7 — Detailed comparison table: Margin is outside the border; Padding is inside the border.

PYQ – Frequently Asked

Q12. List the types of Style Sheets with examples.

Answer: Three types: Inline, Internal, External. See Section 2.2 with complete code for each and comparison table.

5.3 XML / XHTML / DTD PYQs

PYQ – Frequently Asked

Q13. What is XML and why is it used? [REPEATED] (Repeated 3 times)

Answer: XML is Extensible Markup Language used to store and transport data in a human-readable format. See Section 3.1 for full explanation, features, and applications.

PYQ – Frequently Asked

Q14. Define XML Namespaces and explain how they are declared. [REPEATED] (Repeated 2 times)

Answer: See Section 3.3 — Namespaces avoid naming conflicts. Declared with `xmlns:prefix="URI"`. Full examples provided.

PYQ – Frequently Asked

Q15. Compare and contrast DOM Parser and SAX Parser. [REPEATED] (Repeated 3 times)

Answer: See Section 3.7 — Detailed comparison: DOM is tree-based and loads full document; SAX is event-based and streams sequentially.

PYQ – Frequently Asked

Q16. Create an XML document of 10 students containing Roll Number, Marks in 5 Subjects, Total Marks and Percentage. [REPEATED]

Answer: See Section 3.8 — Complete XML listing for 10 students.

PYQ – Frequently Asked

Q17. Define DTD with example. What is the purpose of DTD in XML?

Answer: See Section 3.4 — DTD defines legal XML structure. Internal and External DTD examples provided.

PYQ – Frequently Asked

Q18. How do you convert HTML into XHTML?

Answer: See Section 4.3 — 7 XHTML conversion rules with before/after code.

Chapter 6

Viva & Interview Questions

6.1 HTML Questions

1. **Q: What is HTML?** A: HTML (HyperText Markup Language) is the standard language for creating web pages using tags/elements.
2. **Q: What is the difference between `<b>` and `<strong>`?** A: Both make text bold, but `<strong>` has semantic importance (screen readers emphasize it).
3. **Q: What is an empty element?** A: An element with no closing tag, e.g. `<br>`, `<hr>`, `<img>`.
4. **Q: What is the use of the `alt` attribute in `<img>`?** A: It provides alternate text for screen readers and when the image cannot be displayed.
5. **Q: What is the difference between GET and POST methods in forms?** A: GET appends data to URL (visible, limited size); POST sends data in request body (secure, larger data).
6. **Q: What is a semantic element?** A: Elements that describe their meaning, e.g. `<header>`, `<footer>`, `<article>`, `<nav>`.
7. **Q: What is the purpose of the `<!DOCTYPE html>` declaration?** A: Tells the browser the document is HTML5 and to use standards mode.
8. **Q: What is DHTML?** A: Dynamic HTML = HTML + CSS + JavaScript + DOM for interactive web pages.
9. **Q: What is the difference between `<iframe>` and `<frameset>`?** A: `<iframe>` embeds one page inside another inline; `<frameset>` divides the browser window into multiple frame areas (deprecated).
10. **Q: How do you create an ordered list starting from 5?** A: `<ol start="5">`

6.2 CSS Questions

1. **Q: What is cascading in CSS?** A: When multiple rules apply to an element, CSS applies them in order of specificity and source order.
2. **Q: What is specificity in CSS?** A: Inline (1000) > ID (100) > Class (10) > Element (1).
3. **Q: What is the box model?** A: Content + Padding + Border + Margin = total element space.
4. **Q: What is `box-sizing: border-box`?** A: Makes width/height include padding and border, not just content.
5. **Q: Difference between `display:none` and `visibility:hidden`?** A: `display:none` removes element from layout; `visibility:hidden` hides element but retains space.
6. **Q: What is `z-index`?** A: Controls stacking order of positioned elements.
7. **Q: What is a pseudo-class?** A: Selects elements in a specific state, e.g. `:hover`, `:focus`, `:nth-child`.
8. **Q: What is `position:sticky`?** A: Element behaves as relative until a scroll threshold, then acts as fixed.

6.3 XML Questions

1. **Q: What is the difference between well-formed and valid XML?** A: Well-formed follows XML syntax rules; valid also follows the rules in its DTD or Schema.
2. **Q: Can XML have multiple root elements?** A: No. XML must have exactly one root element.
3. **Q: What are XML entities?** A: `<` (<), `>` (>), `&` (&), `"` ("), `'` (').
4. **Q: What is the advantage of SAX over DOM?** A: SAX uses less memory and is faster for large XML files.
5. **Q: What is the advantage of XSD over DTD?** A: XSD supports data types, namespaces, and is written in XML.
6. **Q: What is a CDATA section?** A: `<![CDATA[...]]>` — content treated as raw text, not parsed as XML.

6.4 XHTML Questions

1. **Q: Why was XHTML introduced?** A: To make HTML more strict and compatible with XML tools.
2. **Q: Are XHTML tags case-sensitive?** A: Yes — all XHTML tags must be in lowercase.
3. **Q: How do you write `<br>` in XHTML?** A: `<br />` — self-closing with a space before slash.
4. **Q: Is DOCTYPE required in XHTML?** A: Yes, DOCTYPE is mandatory in XHTML.

Chapter 7

Unit-I Quick Revision

7.1 Important HTML Tags

Tag	Purpose	Tag	Purpose
`<html>`	Root element	`<head>`	Document metadata
`<body>`	Page content	`<title>`	Browser tab title
`<h1>-<h6>`	Headings	`<p>`	Paragraph
`<a>`	Hyperlink	`<img>`	Image
`<table>`	Table	`<form>`	Form
`<ul>/<ol>`	Lists	`<li>`	List item
`<div>`	Block container	`<span>`	Inline container
`<iframe>`	Inline frame	` `	Line break
`<hr>`	Horizontal rule	`<meta>`	Metadata
`<script>`	JavaScript	`<style>`	CSS
`<header>`	Semantic header	`<footer>`	Semantic footer
`<nav>`	Navigation	`<section>`	Section

7.2 Important CSS Properties

Property	Purpose	Property	Purpose
`color`	Text colour	`background-color`	Background colour
`font-family`	Font name	`font-size`	Text size
`font-weight`	Bold etc.	`text-align`	Alignment
`margin`	Outer spacing	`padding`	Inner spacing
`border`	Border style	`width/height`	Dimensions
`display`	Block/inline	`position`	Positioning

`float`	Float left/right	`z-index`	Stack order
`opacity`	Transparency	`box-shadow`	Shadow effect

7.3 Important XML Terms

Term	Meaning
Well-formed	Follows all XML syntax rules
Valid	Well-formed + conforms to DTD/Schema
DOM	Document Object Model — tree representation
SAX	Simple API for XML — event-driven parsing
DTD	Document Type Definition — validation rules
XSD	XML Schema Definition — advanced validation
Namespace	Avoids naming conflicts (`xmlns:prefix`)
CDATA	Character Data — not parsed by XML

7.4 Important XHTML Rules

1. All tags must be in **lowercase**.
2. All tags must be **properly closed**.
3. Empty elements: `<br />`, `<hr />`, `<img />`.
4. Attribute values must be **quoted**.
5. Tags must be **correctly nested**.
6. **DOCTYPE** declaration is mandatory.
7. Root `<html>` must have **xmlns** attribute.

7.5 Most Probable Exam Questions

Important / Repeated PYQ

1. CSS Selectors — types and examples [REPEATED]
2. CSS Box Model with diagram [REPEATED]
3. DOM Parser vs SAX Parser comparison [REPEATED]
4. Create student registration form with JS validation [REPEATED]
5. XML document for 10 students [REPEATED]
6. HTML vs XML differences [REPEATED]

7. Types of CSS style sheets with examples
8. XML Namespaces — definition and declaration ↻ [REPEATED]
9. DTD with example
10. Converting HTML to XHTML rules
11. Inline, Block, and Semantic elements
12. HTML form elements — all input types

7.6 One-Page Unit I Summary

UNIT I AT A GLANCE

HTML	Structure language for web. Tags, Elements, Attributes. Document = DOCTYPE + html + head + body. Forms, Tables, Lists, Frames.
CSS	Style sheets: Inline > Internal > External. Box Model: Content + Padding + Border + Margin. Selectors: Universal, Type, Class, ID, Pseudo. Positioning: static/relative/absolute/fixed/sticky.
XML	Data storage language. User-defined tags. Case-sensitive. Must have single root. Well-formed vs Valid. DTD/XSD for validation. Namespaces for conflict resolution. Parsers: DOM (tree, random access) vs SAX (event, sequential).
XHTML	Strict HTML+XML. Lowercase tags. All tags closed. Quoted attributes. DOCTYPE mandatory. Namespace in root. Meta tags, Character entities, Frames (deprecated in HTML5).